



ATLAS

ATLAS

Enterprise Architecture for Vibe Coding

The project intelligence and orchestration layer that enables teams of developers and AI coding agents to build large software systems in parallel — without losing architectural control.



From scope to software — at agentic speed.

Vibe coding breaks down as projects and teams scale

AI can generate code quickly. The harder problem is keeping many developers and agents aligned to one system design.

Context fragmentation

Each developer and agent sees only part of the project.

Architecture drift

Local AI decisions slowly diverge from the intended system.

Parallel conflicts

Agents duplicate logic, collide on interfaces and break dependencies.

Scope drift

Fast coding can move faster than requirements and acceptance criteria.

Integration debt

Speed is lost later in review, rework and integration.

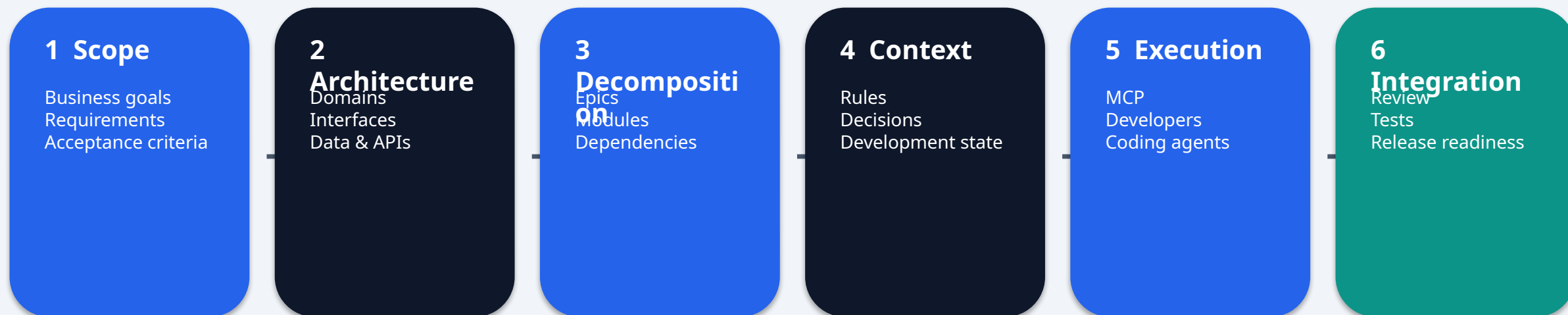
The gap

Coding agents are becoming powerful executors, but large projects still need a shared architectural brain.

ATLAS is designed to make AI-assisted development team-scalable, architecture-aware and continuously contextual.

ATLAS is the control plane for large-scale AI-assisted development

It turns project intent into structured, agent-ready development context.



ATLAS maintains the map. Coding agents execute within clearly defined boundaries.

The result: parallel AI development without turning the codebase into a collection of disconnected prompts.

A well-defined architecture for team-based vibe coding



ATLAS separates project intelligence, context delivery and code execution.

PROJECT INTENT

Scope • workflows • non-functional requirements • acceptance criteria

ATLAS PROJECT INTELLIGENCE

Architecture • domain boundaries • interfaces • dependencies • standards • decisions • task graph

MCP CONTEXT & TOOL LAYER

Context retrieval • repositories • documentation • issue systems • databases • engineering tools

PARALLEL DEVELOPMENT CELLS

Developer + Agent A | Developer + Agent B | Developer + Agent C | Developer + Agent N

CONTROLLED INTEGRATION

Contracts • tests • reviews • dependency validation • release readiness

Core product capabilities

ATLAS gives every developer-agent pair the right context, boundaries and next action.

Architecture Blueprint

Defines system structure, domains, services, data ownership and integration contracts.

Agent-Ready Work Plans

Breaks large scope into executable units with dependencies and definitions of done.

Persistent Project Memory

Retains decisions, conventions, changes and development state across sessions.

MCP Context Delivery

Makes relevant project knowledge and connected tools available to coding agents.

Parallel Work Orchestration

Creates bounded development cells so multiple agents can work concurrently.

Architecture Guardrails

Checks work against interfaces, standards, dependencies and intended system design.

How a team uses ATLAS

A repeatable workflow from product intent to coordinated AI execution.

01 Define

Import or capture project scope, workflows, constraints and acceptance criteria.

02 Architect

Establish technology stack, domains, interfaces, data models and engineering standards.

03 Decompose

Create modules, epics, tasks, dependencies and parallel development lanes.

04 Connect

Link repositories, documents, issue systems and engineering tools through MCP.

05 Build

Developers launch coding agents with bounded, task-specific project context.

06 Control

ATLAS tracks decisions, validates boundaries and keeps the team aligned as the system evolves.

Beachhead ICP: engineering teams scaling AI-assisted delivery

ATLAS is most valuable when the project is too large for prompt-by-prompt development.

Ideal organization

- Software services / digital engineering firms
- SaaS and product engineering teams
- Enterprise application teams
- Legacy modernization specialists

Sweet spot

- 10–200 developers
- 5–50 developers on a shared project
- Multiple AI coding agents
- Medium-to-large business applications
- Parallel feature development

Economic buyer

- CTO / VP Engineering
- Head of Delivery
- Engineering Director
- Chief / Solution Architect

Primary users: architects, tech leads, developers and AI coding agents

Trigger event: “We want AI coding speed across the team, but we cannot sacrifice architecture, scope or integration quality.”

Business value: convert AI coding speed into delivery speed

The objective is not more generated code. It is faster, more predictable delivery of coherent software.



Parallelism

More developer-agent pairs can work simultaneously.



Rework

Shared architecture and context reduce avoidable divergence.



Onboarding time

New developers and agents can understand the project faster.



Consistency

Common standards, interfaces and decisions across modules.



Traceability

Implementation remains connected to scope and acceptance criteria.



Agent portability

Project intelligence is not locked to one coding agent or IDE.

North-star metric Time from approved scope → integrated, production-ready software

ATLAS occupies a new layer in the development stack

It complements — rather than replaces — coding agents and existing engineering tools.

Work management

Jira / Linear / similar

Tracks human work and delivery status

Source control

Git platforms

Stores code, branches, pull requests and history

Coding execution

AI coding agents / IDE agents

Generates, edits, tests and reasons about code

Connectivity

MCP ecosystem

Connects agents to context, tools and data

ATLAS

Architecture + context +
orchestration

Keeps large-scale team execution aligned to the project map

Business model

Land with one project, expand across engineering teams and portfolios.

Team

For product squads and smaller engineering teams.

Pricing basis:

- active developers
- active projects
- MCP integrations

Goal: low-friction adoption

Business

For software companies running multiple client or product projects.

Adds:

- shared templates
- governance
- portfolio visibility
- advanced integrations

Enterprise

For large engineering organizations.

Adds:

- private deployment
- SSO / RBAC
- auditability
- policy controls
- enterprise support

**Expansion
path**

Pilot project → Engineering team → Multi-project rollout → Enterprise standard

Go-to-market: prove ATLAS on real large-scale builds

The fastest route to credibility is measurable delivery improvement on complex projects.

1. Design partners

Recruit 5–10 AI-forward engineering teams with active medium/large projects.

Outcome: validate workflow, architecture and integrations.

2. Lighthouse projects

Run ATLAS on projects with 5+ developers and multiple coding agents.

Outcome: case studies on speed, rework and integration.

3. Developer-led adoption

Templates, MCP connectors, architecture packs and project onboarding.

Outcome: repeatable team deployment.

4. Enterprise expansion

Governance, private deployment, portfolio intelligence and controls.

Outcome: larger ACV and standardization.

Initial wedge

“Take one complex project and make team-based vibe coding work predictably.”

Defensibility: the project map gets smarter as development progresses

ATLAS should build durable value around structured project intelligence — not around access to any single model.

Architecture methodology

A repeatable system for decomposing large applications into agent-buildable domains.

Project knowledge graph

Relationships between requirements, architecture, modules, interfaces, tasks, decisions and code.

Development memory

Long-lived context that survives individual prompts, sessions, agents and developer changes.

Agent-agnostic layer

Customers can change coding agents without rebuilding the project intelligence layer.

Integration depth

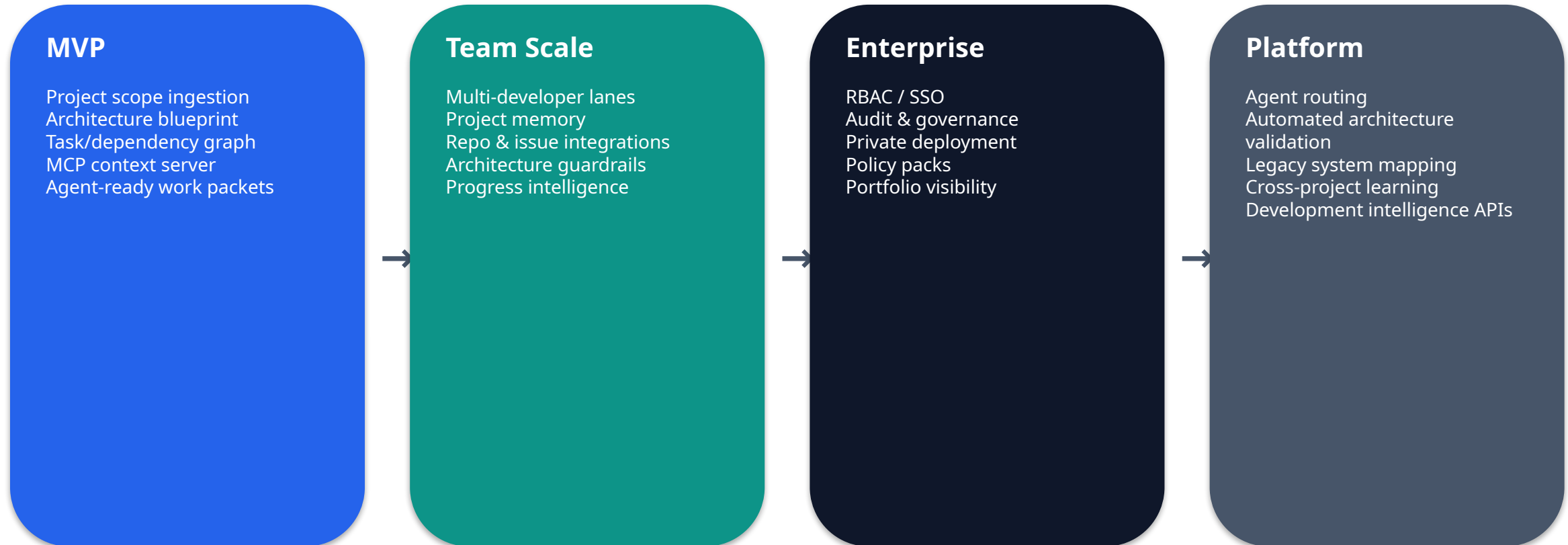
MCP-based access to repositories, documents, issue systems, engineering tools and enterprise data.

Outcome data

Learning from where projects drift, integrate poorly or require rework can improve planning and guardrails.

Product roadmap

Build the control plane in layers, starting with the highest-value team workflow.



Long-term vision: ATLAS becomes the persistent intelligence layer between software intent and AI execution.

ATLAS

Take vibe coding from one developer to an engineering team.

A structured architecture, project intelligence and MCP-powered context layer for building large software systems with developers and AI coding agents.



Map the project. Guide the agents. Ship faster.